

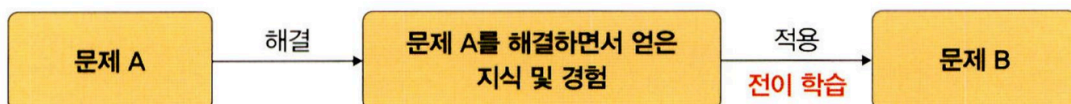
딥러닝 파이토치 교과서 5장 - Part 2 (5.3~)

📄 제출	제출 전
📄 과제 유형	예습
📅 마감일	@2025년 3월 31일
☑ 발표	☐
☑ 완료	☐
📅 주차	4주차

5.3 전이 학습

- CNN 딥러닝 모델을 제대로 훈련시키기 위해서는 많은 양의 데이터가 필요하지만, 현실적으로 충분히 큰 데이터셋을 얻으려면 비용과 시간이 많이 들어 쉽지 않다.
- 이런 어려움을 **전이 학습 (Transfer Learning)**으로 해결했다.
- **전이 학습**
 - 이미지넷처럼 아주 큰 데이터셋을 써서 훈련된 모델의 가중치를 가져와 우리가 해결하려는 과제에 맞게 보정해서 사용하는 것
 - 이때, 아주 큰 데이터셋을 사용해 훈련된 모델을 사전 훈련된 모델 (네트워크)라고 한다. 비교적 적은 수의 데이터를 가지고도 우리가 원하는 과제를 해결할 수 있음

▼ 그림 5-29 전이 학습

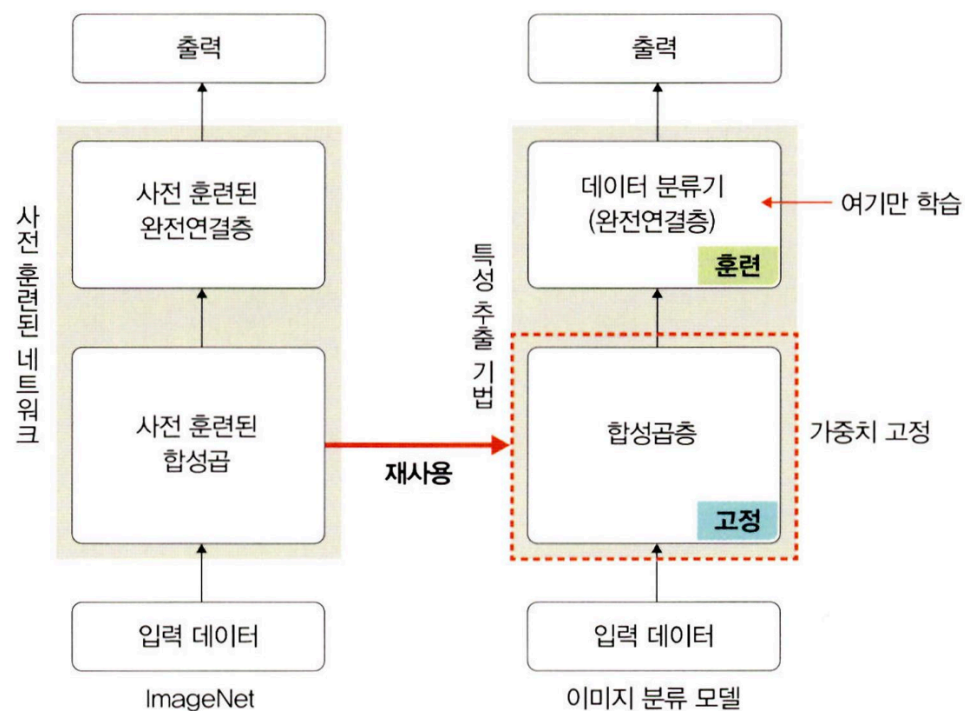


5.3.1 특성 추출 기법

- 이미지넷 데이터셋으로 사전 훈련된 모델을 가져온 후 마지막에 **완전 연결층 부분만** 새로 만들

- 즉, 학습할 때 마지막 완전 연결층 (이미지의 카테고리를 결정하는 부분)만 학습하고, 나머지 계층들은 학습되지 않도록 함
- 특성 추출은 이미지 분류를 위해 합성곱층과 데이터 분류기 (완전 연결층) 두 부분으로 구성됨
 - **합성곱층**: 합성곱층 + 풀링층
 - **데이터 분류기 (완전 연결층)**: 추출된 특성을 입력받아 최종적으로 이미지에 대한 클래스를 분류하는 부분
 - 사전 훈련된 네트워크의 합성곱층(가중치 고정)에 새로운 데이터를 통과시키고, 그 출력을 데이터 분류기에서 훈련시킴
- 사용 가능한 이미지 분류 모델: Xception, Inception V3, ResNet50, VGG16, VGG19, MobileNet

▼ 그림 5-30 특성 추출 기법



```
# 라이브러리 호출
import os
import time
import copy
import glob
import cv2 # Open CV 라이브러리
```

```

import shutil

import torch
import torchvision # 컴퓨터 비전 용도의 패키지
import torchvision.transforms as transforms # 데이터 전처리 패키지
import torchvision.models as models # 다양한 파이토치 네트워크를 사용할 수 있도록
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader

import matplotlib.pyplot as plt

```

이미지 데이터 전처리 방법 정의

```

data_path = 'chap05/data/catanddog/train'
data_path = os.path.join(os.getcwd(), data_path)

transform = transforms.Compose(
    [
        transforms.Resize([256, 256]),
        transforms.RandomResizedCrop(224),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor()
    ]
)

train_dataset = torchvision.datasets.ImageFolder(
    data_path,
    transform = transform
)

train_loader = torch.utils.data.DataLoader(
    train_dataset, batch_size = 32, num_workers = 8, shuffle = True
)

print(len(train_dataset))

```

1. `torchvision.transforms` : 이미지 데이터를 변환하여 모델의 입력으로 사용할 수 있게 변환해 줌

- `transforms.Resize([256, 256])` : 이미지 크기를 256 * 256으로 조정. 합성곱층을 통과하기 위해 이미지 크기를 조정하는 전처리 과정
- `transforms.RandomResizedCrop(224)` : 이미지를 랜덤한 크기 및 비율로 자름, 데이터 확장 (data augmentation) 용도 (이미지를 랜덤한 비율로 자른 후 데이터 크기를 조정)
 - 원본 이미지에서 랜덤한 위치(random position)와 랜덤한 크기(random scale)를 선택해서 잘라냅니다 (crop).
 - 그 잘라낸 이미지를 우리가 지정한 고정된 크기(size)로 리사이즈합니다.

왜 쓰나요?

- **데이터 다양성 확보**: 같은 이미지를 여러 방식으로 잘라서 학습에 사용 → 모델이 위치나 크기에 덜 민감해짐
- **오버피팅 방지**: 모델이 특정 위치나 배경에만 의존하지 않게 함
- **데이터 증강 효과**: 실제 데이터가 적을 때도 다양한 학습 샘플을 만들 수 있음
- `RandomHorizontalFlip` : 이미지를 랜덤하게 수평으로 뒤집음
- `ToTensor` : 이미지 데이터를 텐서로 변환

2. `torchvision.datasets.ImageFolder` : 데이터로더가 데이터를 불러올 대상과 방법을 정의

- `data_path` : 불러올 데이터가 위치한 경로
- `transform` : 이미지 데이터에 대한 전처리

3. `torch.utils.data.DataLoader` : 데이터로더는 데이터를 불러오는 부분으로 앞에서 정의한 `ImageFolder(train_dataset)`을 데이터 로더에 할당함.

- `train_dataset` : 데이터셋 지정
- `batch_size` : 이때 한 번에 불러올 데이터양을 결정
- `num_workers` : 데이터를 불러올 때 하위 프로세스를 몇 개 사용할지 설정. 너무 많은 하위 프로세스를 설정하게 되면 오류가 발생하거나 메모리 부족 현상 발생
 - 하위 프로세스: 데이터를 불러올 때 병렬로 데이터를 로딩하는 데 사용
- `shuffle` : 데이터를 무작위로 shuffle할 것인지 설정

학습에 사용될 이미지 출력


```

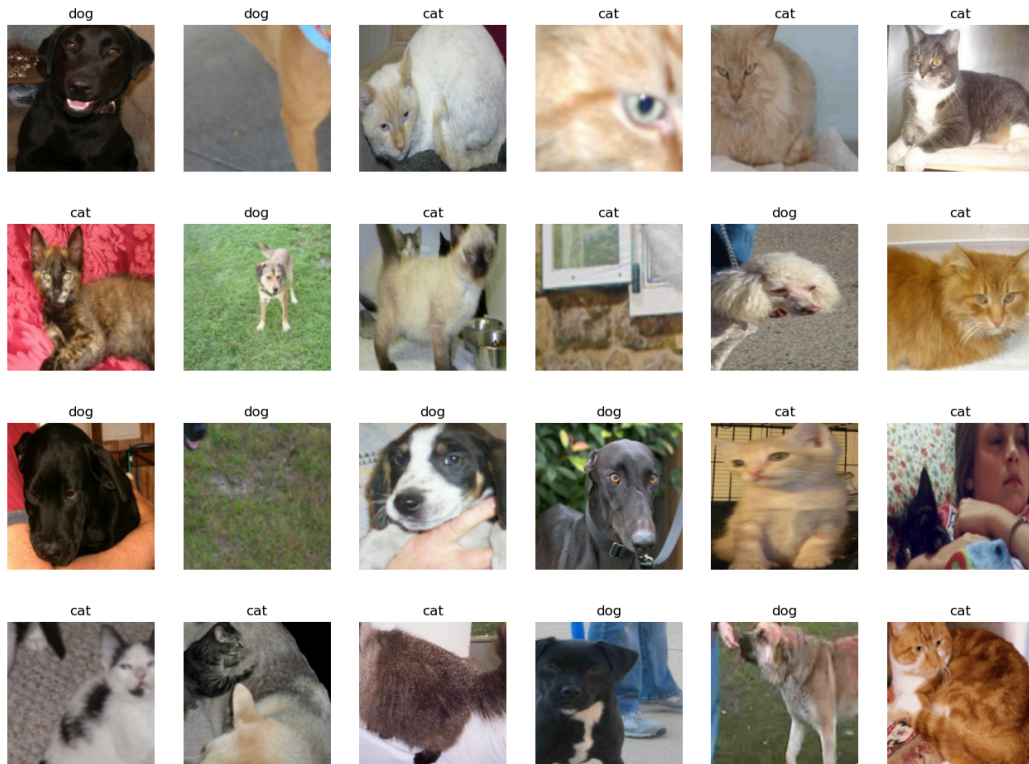
samples, labels = next(iter(train_loader))
classes = {0: 'cat', 1: 'dog'} # 개와 고양이에 대한 클래스로 구성

fig = plt.figure(figsize = (16, 24))

for i in range(24): # 24 개의 이미지 데이터 출력
    a = fig.add_subplot(4, 6, i + 1) # column, row, index
    a.set_title(classes[labels[i].item()]) # 레이블 정보(클래스)를 함께 출력
    a.axis('off')
    a.imshow(np.transpose(samples[i].numpy(), (1, 2, 0)))

plt.subplots_adjust(bottom = 0.2, top = 0.6, hspace = 0)

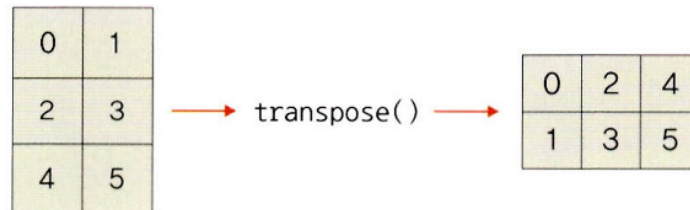
```



1. `next(iter(train_loader))` : 반복자(iterator, for 구문과 같은 효과)를 사용하려면 `iter()` 와 `next()` 가 필요하다
 - `iter()` : 전달된 데이터의 반복자를 꺼내 반환
 - `next()` : 그 반복자가 다음에 출력해야할 요소를 반환
 - `iter()` 로 반복자를 구하고, 그 반복자를 `next()` 에 전달하여 차례대로 꺼냄

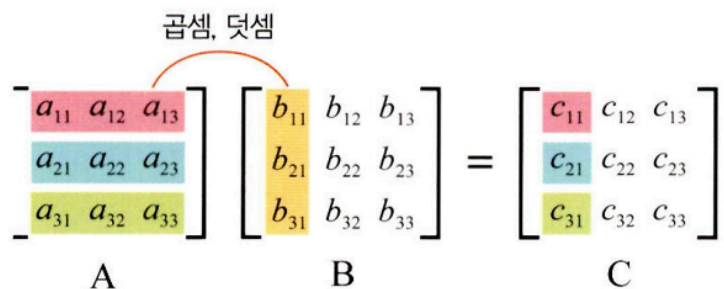
- 즉, iterator인 train_loader에서 samples와 labels를 순차적으로 꺼내서 저장함
2. `np.transpose(samples[i].numpy(), (1, 2, 0))` : (224, 224, 3)과 같은 형태로 변환
- PyTorch는 이미지 데이터를 (C, H, W) 형태로 저장하지만, matplotlib은 (W, H, C)의 numpy 이미지 형식을 기대함
 - `np.transpose` : 행과 열을 바꿔서 행렬의 차원을 변경

▼ 그림 5-33 transpose()



- 행렬의 차원을 바꾸는 이유는 행렬의 내적 연산을 수행하기 위해서. A와 B 행렬의 내적 연산을 할 때 A의 행과 B의 열이 맞아야 함

▼ 그림 5-34 행렬의 내적



$$\begin{bmatrix} c_{11} \\ c_{21} \\ c_{31} \end{bmatrix} = \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} + a_{13}b_{31} \\ a_{21}b_{11} + a_{22}b_{21} + a_{23}b_{31} \\ a_{31}b_{11} + a_{32}b_{21} + a_{33}b_{31} \end{bmatrix}$$

사전 훈련된 모델 내려받기

```
resnet18 = models.resnet18(pretrained = True) # pretrained = True: 사전 학습된
```

- **resnet18**: 50 개의 계층으로 구성된 합성곱 신경망

- ImageNet 데이터 베이스의 100만개가 넘는 영상을 이용해 훈련된 신경망으로 전이 학습에 사용되도록 사전 훈련된 모델을 제공
- 하지만, 입력 제약이 매우 크고 충분한 메모리가 없으면 학습 속도가 느릴 수 있음

사전 훈련된 모델의 파라미터 학습 유무 지정

```
def set_parameters_requires_grad(model, feature_extracting = True):
    if feature_extracting:
        for param in model.parameters():
            param.requires_grad = False

set_parameters_requires_grad(resnet18)
```

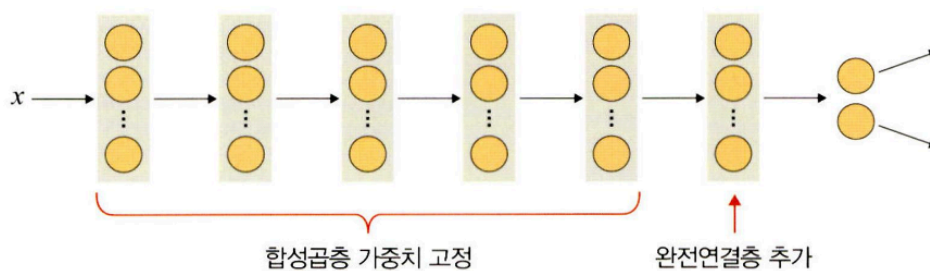
- `param.requires_grad = False` : 역전파 파라미터들에 대한 변화를 계산할 필요가 없음을 나타냄
- 모델의 일부(합성곱층과 풀링층)를 고정하고 나머지를 학습하고자 할 때 사용

ResNet18에 완전 연결층 추가

```
resnet18.fc = nn.Linear(512, 2) #2: 클래스가 2 개
```

- 완전 연결층을 추가해 개와 고양이 클래스를 분류하는 용도로 사용

▼ 그림 5-36 ResNet18에 완전연결층 추가



모델의 파라미터 값 확인

```
for name, param in resnet18.named_parameters():
    if param.requires_grad:
        print(name, param.data)
```

```

fc.weight tensor([[-0.0245, -0.0134, 0.0187, ..., 0.0019, -0.0130, 0.0220],
                  [-0.0326, 0.0262, -0.0026, ..., 0.0366, 0.0179, 0.0144]])
fc.bias tensor([-0.0151, -0.0275])

```

모델 객체 생성 및 손실 함수 정의

```

warnings.filterwarnings(action='ignore')
model = models.resnet18(pretrained = True) # 모델 객체 생성

for param in model.parameters():
    param.requires_grad = False # 모델의 합성곱층 가중치 고정

model.fc = torch.nn.Linear(512, 2)
for param in model.fc.parameters():
    param.requires_grad = True # 완전 연결층은 학습

optimizer = torch.optim.Adam(model.fc.parameters())
cost = torch.nn.CrossEntropyLoss() # 손실 함수 정의
print(model)

```

```

ResNet(
  (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU(inplace=True)
  (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=False)
  (layer1): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
    (1): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
  (layer2): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(64, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
    ...
  )
  (avgpool): AdaptiveAvgPool2d(output_size=(1, 1))
  (fc): Linear(in_features=512, out_features=2, bias=True)
)

```

모델 학습을 위한 함수 생성

```

def train_model(model, dataloaders, criterion, optimizer,
                device, num_epochs = 13, is_train = True):

```

```

since = time.time() # 컴퓨터의 현재 시각
acc_history = [] # 각 epoch마다 정확도를 저장하는 리스트.
loss_history = [] # 각 epoch마다 손실 값을 저장하는 리스트.
best_acc = 0.0 # 지금까지의 최고 정확도를 저장하는 변수.

for epoch in range(num_epochs): # 에포크(13)만큼 반복
    print("Epoch {}/{}".format(epoch, num_epochs - 1)) # 현재 몇 번째 epoch인
    print('-' * 10)

    running_loss = 0.0
    running_corrects = 0

    for inputs, labels in dataloaders: # 데이터로더에 전달된 데이터만큼 반복 # dat
        inputs = inputs.to(device)
        labels = labels.to(device)

        model.to(device)
        optimizer.zero_grad() # 기울기를 0으로 설정 (기울기 초기화)
        outputs = model(inputs) # 순전파 학습
        loss = criterion(outputs, labels) # 손실 함수 계산
        _, preds = torch.max(outputs, 1) # 가장 높은 확률을 가진 클래스 인덱스를 예
        loss.backward() # 역전파 수행
        optimizer.step() # 모델의 가중치를 업데이트

        running_loss += loss.item() * inputs.size(0) # 출력 결과와 레이블의 오차를
        running_corrects += torch.sum(preds == labels.data) # 출력 결과와 레이

    epoch_loss = running_loss / len(dataloaders.dataset) # 평균 오차 계산
    epoch_acc = running_corrects.double() / len(dataloaders.dataset) # 평균

    print(f'Loss: {epoch_loss:.4f} Acc {epoch_acc:.4f}')

    if epoch_acc > best_acc:
        best_acc = epoch_acc # 지금까지 최고 정확도 계산

    acc_history.append(epoch_acc.item())
    loss_history.append(epoch_loss)
    torch.save(model.state_dict(), os.path.join(data_path,

```

```

        '{0:0=2d.pth}'.format(epoch))) # 모델 재사용을 위해
    print()

    time_elapsed = time.time - since # 실행 시간 계산
    print('Training complete in {:.0f}m {:.0f}s'.format(time_elapsed//60, time_elapsed%60))
    print('Best Acc: {4f}'.format(best_acc))
    return acc_history, loss_history # 모델의 정확도와 오차를 반환

```

파라미터 학습 결과를 옵티마이저에 전달

- ResNet18에 추가된 완전 연결층은 학습을 하도록 설정하고, 학습을 통해 얻어진 파라미터를 옵티마이저 전달해서 최종적으로 모델 학습에 사용

```

params_to_update = []
for name, param in resnet18.named_parameters():
    if param.requires_grad == True:
        params_to_update.append(param) # 파라미터 학습 결과 저장
        print("\t", name)

optimizer = optim.Adam(params_to_update) # 학습 결과를 옵티마이저에 전달

```

모델 학습

```

device = torch.device("mps" if torch.backends.mps.is_available() else "cpu")
criterion = nn.CrossEntropyLoss() # 손실 함수 지정
train_acc_hist, train_loss_hist = train_model(resnet18, train_loader, criterion,
                                                optimizer, device)

```

- 모델 훈련을 위해 전달되는 파라미터: 모델, 학습 데이터, 손실 함수, 옵티마이저, 장치

```

Epoch 0/12
-----
Loss: 0.1373 Acc 0.9403

Epoch 1/12
-----
Loss: 0.1562 Acc 0.9273

Epoch 2/12
-----
Loss: 0.2132 Acc 0.9039

Epoch 3/12
-----
Loss: 0.1258 Acc 0.9351

Epoch 4/12
-----
Loss: 0.1366 Acc 0.9481

Epoch 5/12
-----
Loss: 0.1396 Acc 0.9481

Epoch 6/12
...
Loss: 0.1219 Acc 0.9610

Training complete in 10m 34.740052s
Best Acc: 0.9610

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

```

96.10 %로 상당히 높은 정확도를 보여줌

테스트 데이터 호출 및 전처리

```

test_path = 'chap05/data/catanddog/test'
test_path = os.path.join(os.getcwd(), test_path)

transform = transform.Compose([
    transforms.Resize(224),
    transforms.CenterCrop(224),
    transforms.ToTensor()
])

test_dataset= torchvision.datasets.ImageFolder(
    root = test_path,
    transform = transform
)

test_loader = torch.utils.data.DataLoader(

```

```

    test_dataset, batch_size = 32, num_workers = 1, shuffle = True
)

print(len(test_dataset))

```

98

테스트 데이터 평가 함수 생성

```

def eval_model(model, dataloaders, device):
    since = time.time()
    acc_history = []
    best_acc = 0.0

    saved_models = glob.glob('/chap05/data/cantanddog/' + '*.pth')
    saved_models.sort() # 불러온 .pth 파일 정렬
    print('saved_model', saved_models)

    for model_path in saved_models:
        print("Loading model", model_path)

        model.load_state_dict(torch.load(model_path))
        model.eval()
        model.to(device)
        running_corrects = 0

        for inputs, labels in dataloaders: # 테스트 반복
            inputs = inputs.to(device)
            labels = labels.to(device)

            with torch.no_grad(): # autograd를 사용하지 않겠다는 의미
                outputs = model(inputs) # 데이터를 모델에 적용한 결과를 outputs에 저장

            _, preds = torch.max(outputs.data, 1)
            preds[preds >= 0.5] = 1
            preds[preds < 0.5] = 0
            running_corrects += preds.eq(labels.cpu()).int().sum()

```



```

epoch_acc = running_corrects.float() / len(dataloaders.dataset) # 테스트 데이터셋의 정확도
print(f'Acc: {epoch_acc:.4f}')

if epoch_acc > best_acc:
    best_acc = epoch_acc
acc_history.append(epoch_acc.item())
print()

time_elapsed = time.time() - since
print(f"Validation complete in {time_elapsed // 60:.0f}m {time_elapsed % 60:.0f}s")
print(f"Best Acc: {best_acc:.4f}")

return acc_history # 계산된 정확도 반환

```

1. `glob.glob('/chap05/data/cantanddog/' + '*.pth')`
 - `glob` 은 현재 디렉터리에서 원하는 파일(pth 확장자)들만 추출하여 가져올 때 사용
2. `torch.max(outputs.data, dim = 1)` : 주어진 텐서 배열의 최댓값이 들어있는 index를 반환하는 함수
 - `dim=1` 은 **열 방향**, 즉 각 샘플의 클래스들 중에서 최대값을 찾는다는 뜻입니다.
3. `preds.eq(labels.cpu()).int().sum()`
 - `preds.eq(labels.cpu())` : preds 배열과 labels가 일치하는지 검사
 - `.sum()` 은 모델의 예측 결과와 정답이 일치하는 것들의 개수 합을 숫자로 출력

테스트 데이터를 평가 함수에 적용

```
val_acc_hist = eval_model(resnet18, test_loader, device)
```

```

saved_model [' /Users/janghyoin/Desktop/2025/Euron/예습 과제/Week4/chap05/data/catanddog/train/00
Loading model /Users/janghyoin/Desktop/2025/Euron/예습 과제/Week4/chap05/data/catanddog/train/00
Acc: 0.9490

Loading model /Users/janghyoin/Desktop/2025/Euron/예습 과제/Week4/chap05/data/catanddog/train/01
Acc: 0.9490

Loading model /Users/janghyoin/Desktop/2025/Euron/예습 과제/Week4/chap05/data/catanddog/train/02
Acc: 0.9082

Loading model /Users/janghyoin/Desktop/2025/Euron/예습 과제/Week4/chap05/data/catanddog/train/03
Acc: 0.9490

Loading model /Users/janghyoin/Desktop/2025/Euron/예습 과제/Week4/chap05/data/catanddog/train/04
Acc: 0.9388

Loading model /Users/janghyoin/Desktop/2025/Euron/예습 과제/Week4/chap05/data/catanddog/train/05
Acc: 0.9388

Loading model /Users/janghyoin/Desktop/2025/Euron/예습 과제/Week4/chap05/data/catanddog/train/06
Acc: 0.9490

Loading model /Users/janghyoin/Desktop/2025/Euron/예습 과제/Week4/chap05/data/catanddog/train/07
Acc: 0.9388
...
Acc: 0.9490

Validation complete in 1m 40
Best Acc: 0.9490

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

```

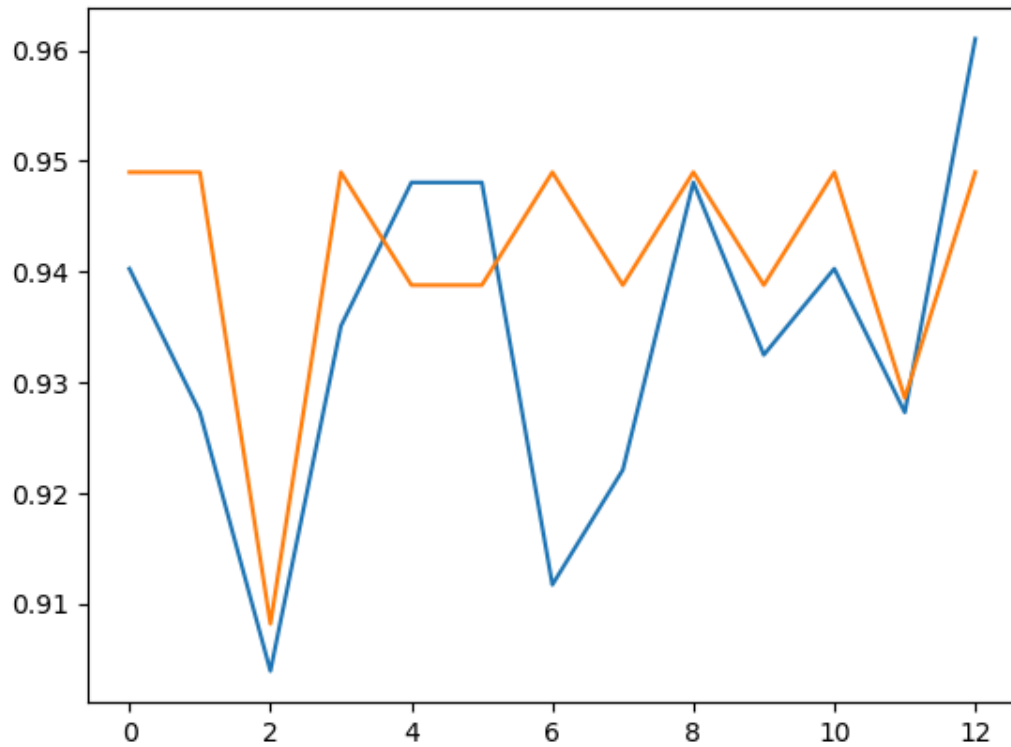
테스트 데이터 역시 94 % 정도의 높은 정확도를 보임.

훈련과 테스트 데이터의 정확도를 그래프로 확인

```

plt.plot(train_acc_hist)
plt.plot(val_acc_hist)
plt.show()

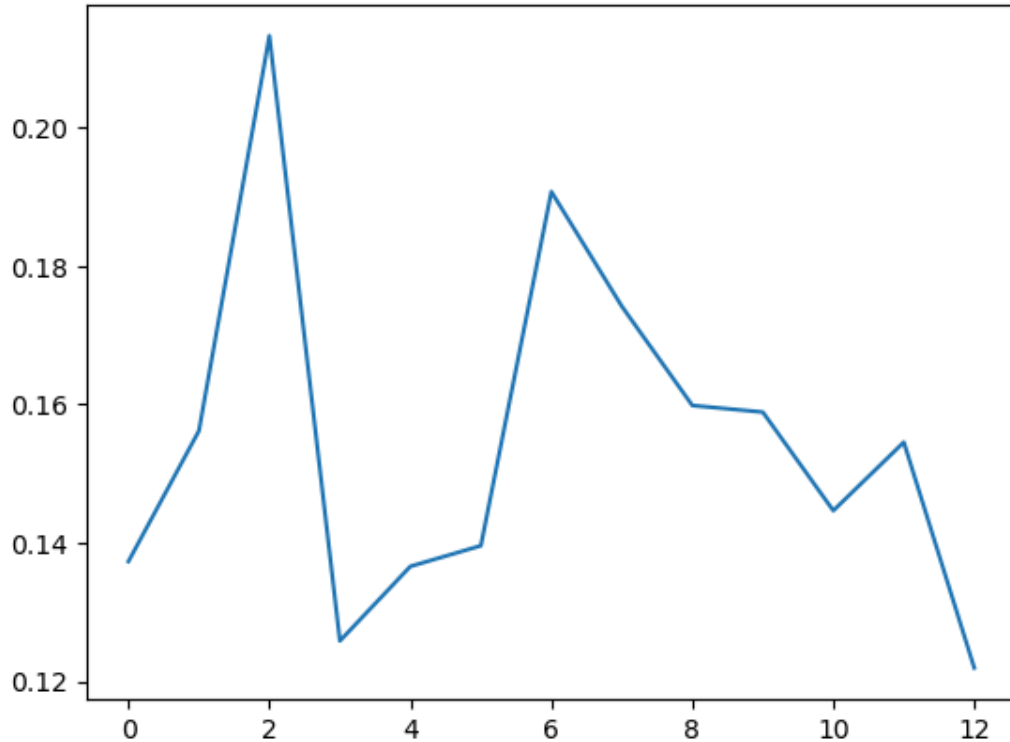
```



- 훈련과 테스트 데이터 모두 에포크가 진행될 수록 정확도가 높아지는 경향을 보임

훈련 데이터의 오차에 대한 그래프 확인

```
plt.plot(train_loss_hist)
plt.show()
```



에포크가 진행될 때마다 오차가 낮아지고 학습이 잘 됨

예측 이미지 출력을 위한 전처리 함수

```
def im_convert(tensor):
    image = tensor.clone().detach().numpy()
    image = image.transpose(1, 2, 0)
    image = image * (np.array((0.5, 0.5, 0.5)) + np.array((0.5, 0.5, 0.5)))
    image = image.clip(0, 1)
    return image
```

1. `tensor.clone().detach()`

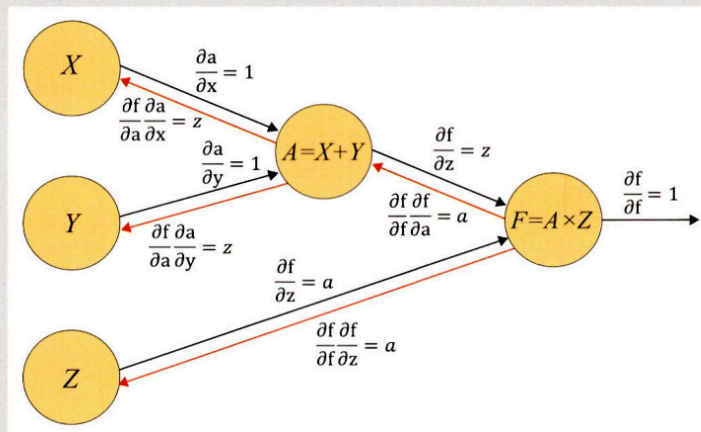
- `tensor.clone()` 기존 텐서의 내용을 복사한 텐서 생성
- `tensor.clone().detach()` : 기존 텐서를 복사한 새로운 텐서를 생성하지만, 기울기에는 영향 x

구분	메모리	계산 그래프 상주 유무
<code>tensor.clone()</code>	새롭게 할당	계산 그래프에 계속 상주
<code>tensor.detach()</code>	공유해서 사용	계산 그래프에 상주하지 않음
<code>tensor.clone().detach()</code>	새롭게 할당	계산 그래프에 상주하지 않음

Note ≡ 계산 그래프

계산 그래프(computational graph)는 이미 2장에서 살펴보았지만 리마인드를 위해 다시 살펴보겠습니다. 계산 그래프란 계산 과정을 그래프로 나타낸 것입니다. 그래프는 여러 개의 노드(node)와 그 노드들을 연결하는 선인 에지(edge)로 구성됩니다.

▼ 그림 5-39 계산 그래프



계산 그래프를 사용하는 이유는 두 가지입니다.

- 국소적 계산이 가능합니다. 국소적 계산이 가능하다는 의미는 그림 5-39에서 Z 값이 변경되었다면 X, Y 계산 결과를 그대로 유지한 채로 바뀐 Z의 연산이 필요한 $F=A \times Z$ 만 계산하면 됩니다.
- 역전파를 통한 미분 계산이 편리합니다. 그림 5-39의 주황색 선이 역전파를 구하는 과정을 보여 주는데 연쇄 법칙(chain rule)을 이용하여 빠르고 간편하게 미분을 계산할 수 있습니다.

2. `image.clip(0, 1)` : image 데이터를 0과 1 사이의 값으로 제한

- `clip()` 입력값이 주어진 범위를 벗어날 때 입력 값을 특정 범위로 제한시키기 위해 사용

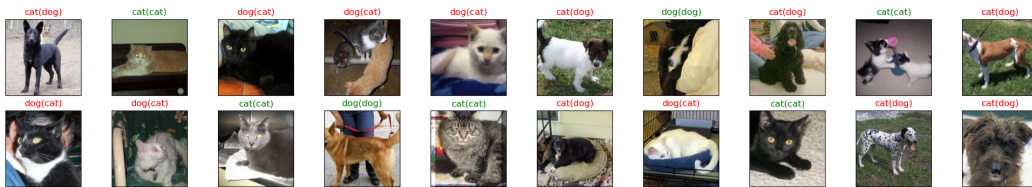
개와 고양이 예측 결과 출력

```
classes = {0: 'cat', 1: 'dog'}

dataiter = iter(test_loader)
images, labels = dataiter.next()
output = model(images)
_, preds = torch.max(output, 1)
```

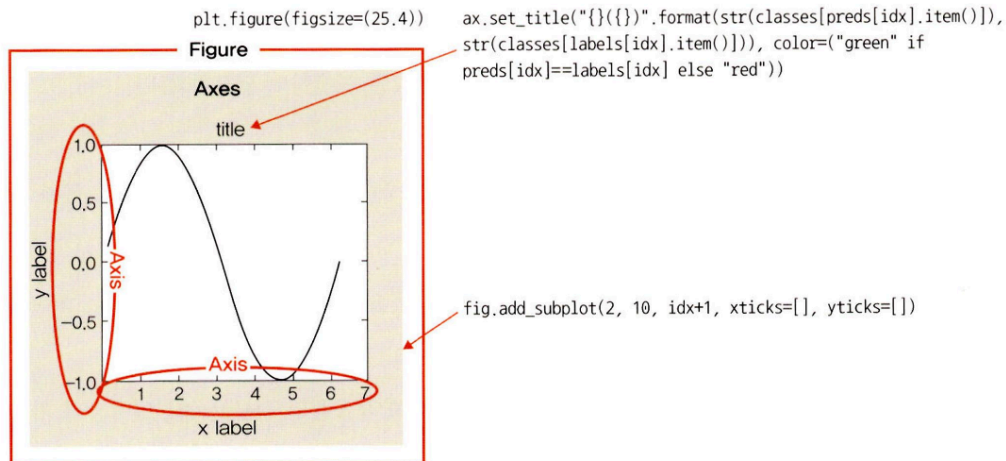
```
fig = plt.figure(figsize = (25, 4))
for idx in np.arange(20):
    ax = fig.add_subplot(2, 10, idx + 1, xticks = [], yticks = [])
    plt.imshow(im_convert(images[idx]))
    a.set_title("{}{}".format(str(classes[preds[idx].item()]),
                              str(classes[labels[idx].item()])),
               color = ('green' if preds[idx] == labels[idx] else 'red'))

plt.show()
plt.subplots_adjust(bottom = 0.2, top = 0.6, hspace = 0)
```



1. `fig.add_subplot(2, 10, idx + 1, xticks = [], yticks = [])`
 - `add_subplot` : 한 화면에 여러 개의 이미지를 담기 위해 사용
 - `(2, 10)` : 행, 열
 - `idx + 1` : 인덱스, 행과 열을 기준으로 순차적으로 이미지 출력
 - `xticks = [], yticks = []` : 틱을 삭제
2. `classes[preds[idx].item()]` : `preds[idx].item()` 값이 `classes`로 정의된 '0'과 '1' 중 어떤 값을 갖는지 판별하겠다는 의미 (0: 고양이, 1: 강아지)
3. `plt.subplots_adjust(bottom = 0.2, top = 0.6, hspace = 0)` : Figure 안에서 subplot의 위치를 조정할 때 사용
 - a. `left, top, bottom, right` 으로 이미지 위치 조정
 - b. `hspace, wspace` 로 서브플롯 간의 간격 조정 가능

▼ 그림 5-42 서브플롯

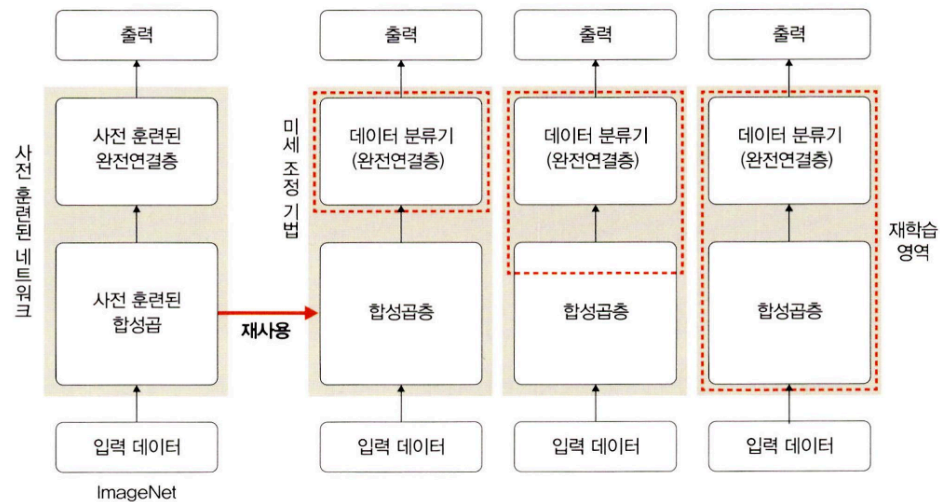


5.3.2 미세 조정 기법

- 미세 조정 기법은 특성 추출 기법에서 더 나아가 사전 훈련된 모델과 합성곱층, 데이터 분류기의 가중치를 업데이트하여 훈련시키는 방식입니다.
 - 특성 추출은 목표 특성을 잘 추출했다는 전제하에 좋은 성능을 낼 수 있지만, 특성이 잘 못 추출되었다면 미세 조정 기법으로 새로운 이미지 데이터를 사용해 네트워크의 가중치를 업데이트해서 특성을 다시 추출할 수 있음
 - 즉, 학습된 모델을 목적에 맞게 재학습시키거나 학습된 가중치의 일부를 재학습시키는 것
- 미세 조정 기법은 사전 훈련된 네트워크를 미세 조정하여 분석하려는 데이터셋에 잘 맞도록 모델의 파라미터를 조절하는 기법

	데이터셋 크기: 큼	데이터셋 크기: 작음
사전 훈련 모델과 유사성: 큼	합성곱층의 뒷부분과 데이터 분류기를 학습시킴	합성곱층 일부분과 데이터 분류기 학습.
사전 훈련 모델과 유사성: 작음	모델 전체 재학습	데이터 분류기만 학습, 완전 연결층에서만 미소 조정 기법 적용

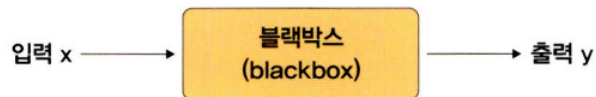
▼ 그림 5-44 미세 조정 기법



5.4 설명 가능한 CNN

- 딥러닝 처리 결과를 사람이 이해할 수 있는 방식으로 제시하는 기술
- CNN은 **블랙 박스**와 같아 내부에서 어떻게 동작하는지 설명하기 어려움
- CNN 결과를 신뢰하기 위해 CNN 처리 과정을 시각화해야 할 필요성이 있음

▼ 그림 5-45 CNN의 블랙박스



- CNN을 구성하는 각 중간 계층부터 최종 분류까지 입력된 이미지에서 특성이 어떻게 추출되고 학습하는지 시각적으로 설명할 수 있어야 결과에 대한 신뢰성을 얻을 수 있음
- CNN의 시각화 방법에는 필터에 대한 시각화와 특성 맵에 대한 시각화가 있지만, 특성 맵에 대한 시각화만 다룸

5.4.1 특성 맵 시각화

- 특성 맵은 입력 이미지 또는 다른 특성 맵처럼 **필터를 입력에 적용한 결과**
- 특정 입력 이미지에 대한 특성맵을 시각화한다는 의미는 **특성 맵에서 입력 특성을 감지하는 방법을 이해할 수 있도록 돕는 것**
- PIL (Python Image Library) 필요

- 다양한 이미지 파일 형식을 지원하며, 강력한 이미지 처리와 그래픽 기능을 제공하는 이미지 프로세싱 라이브러리

설명 가능한 네트워크 생성

```
class XAI(nn.Module):
    def __init__(self, num_classes=2):
        super(XAI, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=3, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True), # inplace=True는 기존의 데이터를 연산의 결과값으로 덮어쓰는 것
            nn.Dropout(0.3),
            nn.Conv2d(64, 64, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),

            nn.Conv2d(64, 128, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Conv2d(128, 128, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),

            nn.Conv2d(128, 256, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(256),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Conv2d(256, 256, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(256),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Conv2d(256, 256, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(256),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Conv2d(256, 256, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(256),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),
```

```

nn.Conv2d(256, 512, kernel_size=3, padding=1, bias=False),
nn.BatchNorm2d(512),
nn.ReLU(inplace=True),
nn.Dropout(0.4),
nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
nn.BatchNorm2d(512),
nn.ReLU(inplace=True),
nn.Dropout(0.4),

nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
nn.BatchNorm2d(512),
nn.ReLU(inplace=True),
nn.MaxPool2d(kernel_size=2, stride=2),

nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
nn.BatchNorm2d(512),
nn.ReLU(inplace=True),
nn.Dropout(0.4),
nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
nn.BatchNorm2d(512),
nn.ReLU(inplace=True),
nn.Dropout(0.4),
nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
nn.BatchNorm2d(512),
nn.ReLU(inplace=True),
nn.MaxPool2d(kernel_size=2, stride=2),
)

self.classifier = nn.Sequential(
    nn.Linear(512, 512, bias=False),
    nn.Dropout(0.5),
    nn.BatchNorm1d(512),
    nn.ReLU(inplace=True),
    nn.Dropout(0.5),
    nn.Linear(512, num_classes)
)

```

```
def forward(self, x):
    x = self.features(x)
    x = x.view(-1, 512)
    x = self.classifier(x)
    return F.log_softmax(x, dim=1)
```

- `F.log_softmax(x, dim=1)` : 신경망 말단의 결과값들을 확률 개념으로 해석하기 위해 소프트맥스 함수 결과에 log 값을 취한 연산
 - 그냥 softmax 함수보다 기울기 소멸 문제에 robust

$$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_{j=1} e^{x_j}}$$

$$\begin{aligned} \text{logsoftmax} &= \log\left(\frac{e^{x_i}}{\sum_{j=1} e^{x_j}}\right) \\ &= x_i - \log\left(\sum_{j=1} e^{x_j}\right) \end{aligned}$$

모델 객체화

```
model = XAI()
model.to(device)
model.eval()
```

```

XAI(
  (features): Sequential(
    (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), bias=False)
    (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU(inplace=True)
    (3): Dropout(p=0.3, inplace=False)
    (4): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (5): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (6): ReLU(inplace=True)
    (7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (8): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (9): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (10): ReLU(inplace=True)
    (11): Dropout(p=0.4, inplace=False)
    (12): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (13): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (14): ReLU(inplace=True)
    (15): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (16): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (17): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (18): ReLU(inplace=True)
    (19): Dropout(p=0.4, inplace=False)
    (20): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (21): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (22): ReLU(inplace=True)
    ...
    (3): ReLU(inplace=True)
    (4): Dropout(p=0.5, inplace=False)
    (5): Linear(in_features=512, out_features=2, bias=True)
  )
)

```

특성 맵의 결과를 확인할 수 있는 함수 정의

- 합성곱층을 입력 이미지와 필터를 연산하여 얻은 결과
- 합성곱층에서 입력과 출력을 알 수 있다면 특성 맵에 대한 값들을 확인할 수 있음

```

class LayerActivations:
    features = []
    def __init__(self, model, layer_num):
        self.hook = model[layer_num].register_forward_hook(self.hook_fn)

    def hook_fn(self, module, input, output):
        self.features = output.detach().numpy()

    def remove(self):
        self.hook.remove()

```

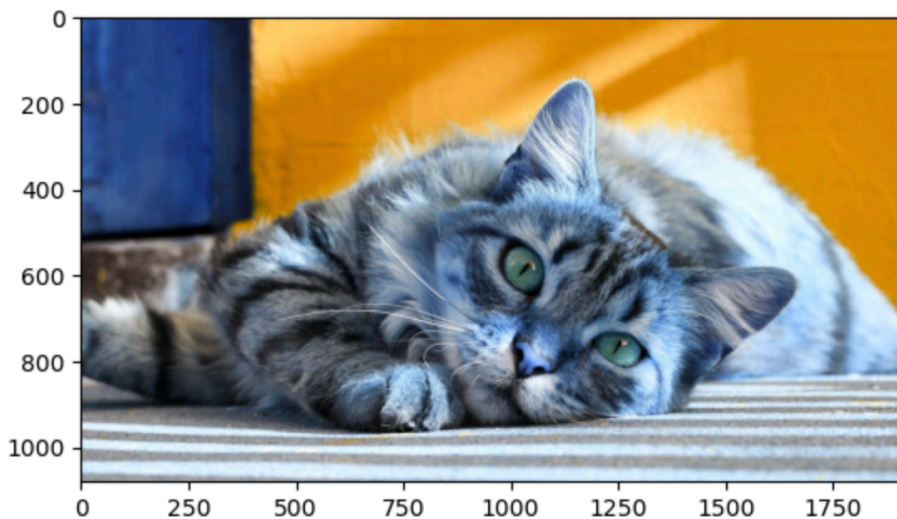
- `model[layer_num].register_forward_hook(self.hook_fn)` : PyTorch는 매 계층마다 print문을 사용하지 않더라도 hook 기능을 사용해 각 계층의 활성화 함수 및 기울기 값을 확인할 수 있음

음 → `register_forward_hook` : 순전파 중에 각 네트워크 모듈의 입력 및 출력을 가져오는 것

이미지 호출

```
img = cv2.imread('chap05/data/cat.jpg')
plt.imshow(img)
img = cv2.resize(img, (100, 100), interpolation = cv2.INTER_LINEAR)
img = ToTensor()(img).unsqueeze(0)
print(img.shape)
```

`torch.Size([1, 3, 100, 100])`



1. `cv2.resize(img, (100, 100), interpolation = cv2.INTER_LINEAR)`
 - `img` : 변경할 이미지 파일
 - `(100, 100)` : 변경될 이미지 크기를 (너비, 높이)로 지정
 - `interpolation` : 보간법
 - 이미지 크기를 변경할 경우 변형된 이미지의 픽셀을 추정해서 값을 할당해야함. 이미지 비율을 변경하면 존재하지 않는 영역에 새로운 픽셀값을 매핑하거나 존재하는 픽셀들을 압축해서 새로운 값을 할당해야함. 이러한 상황을 피하고자 이미지 상에 존재하는 픽셀 데이터 (x_i, y_i) 들에 대해 근사 함수 $f(x, y)$ 를 적용해서 새로운 픽셀 값을 구하는 것
 - 추정해야하는 픽셀을 보간법을 이용해 값을 할당함
2. `ToTensor()(img).unsqueeze(dim = 0)` 에서 `unsqueeze(dim = 0)` 는 1차원 데이터를 생성하는 함수 (이미지 데이터 → 텐서 → 1차원으로 변경)

```

x2 = x1.unsqueeze(dim=0) ----- [3, 10, 64] -> [1, 3, 10, 64]
print(x2.shape)
print('-----')
x3 = x1.unsqueeze(dim=1) ----- [3, 10, 64] -> [3, 1, 10, 64]
print(x3.shape)

```

Conv2d 특성맵 확인

```

result = LayerActivations(model.features, 0) # 0번째 conv2d 특성 맵 확인
model(img).to(device)
activations = result.features

```

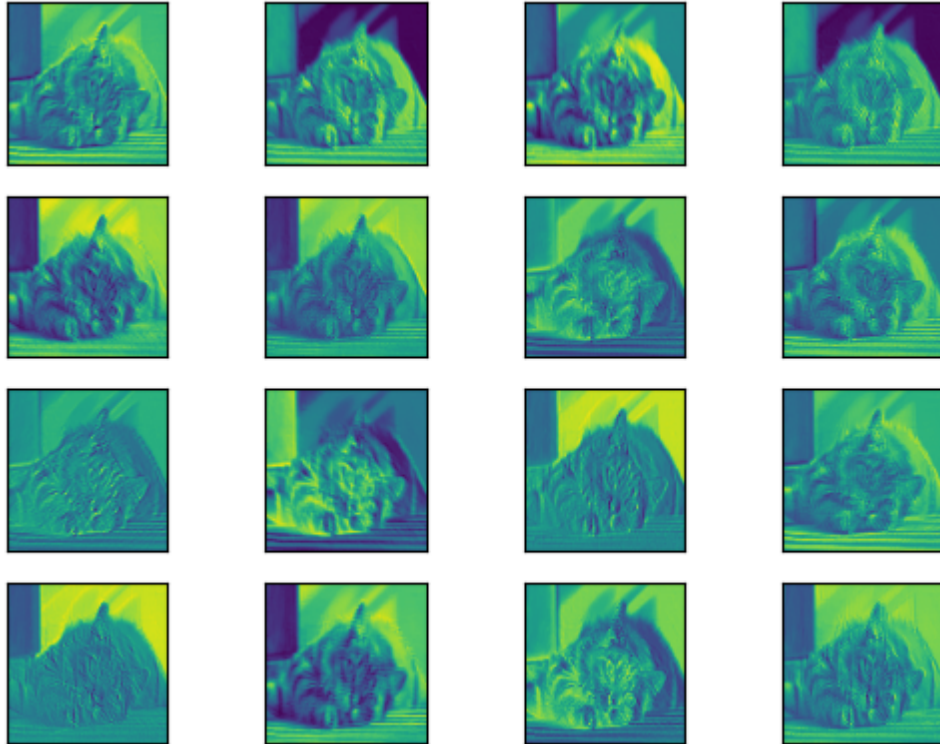
특성 맵 확인

```

fig, axes = plt.subplots(4, 4)
fig = plt.figure(figsize = (12, 8))
fig.subplots_adjust(left = 0, right = 1, bottom = 0, top = 1, hspace = 0.05, wspace = 0.05)

for row in range(4):
    for column in range(4):
        axis = axes[row][column]
        axis.get_xaxis().set_ticks([])
        axis.get_yaxis().set_ticks([])
        axis.imshow(activations[0][row * 10 + column])
plt.show()

```



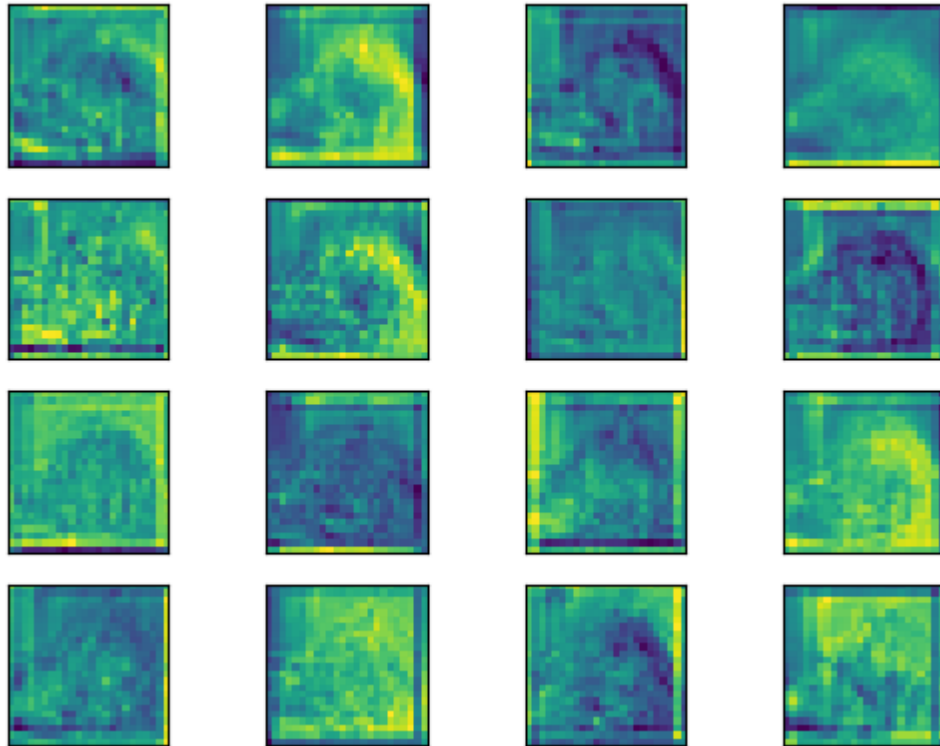
20번째 계층에 대한 특성 맵

```
result = LayerActivations(model.features, 20)
model(img)
activations = result.features
```

특성 맵 확인

```
fig, axes = plt.subplots(4, 4)
fig = plt.figure(figsize = (12, 8))
fig.subplots_adjust(left = 0, right =1, bottom = 0, top =1, hspace = 0.05, wspace = 0.05)

for row in range(4):
    for column in range(4):
        axis = axes[row][column]
        axis.get_xaxis().set_ticks([])
        axis.get_yaxis().set_ticks([])
        axis.imshow(activations[0][row * 10 + column])
plt.show()
```



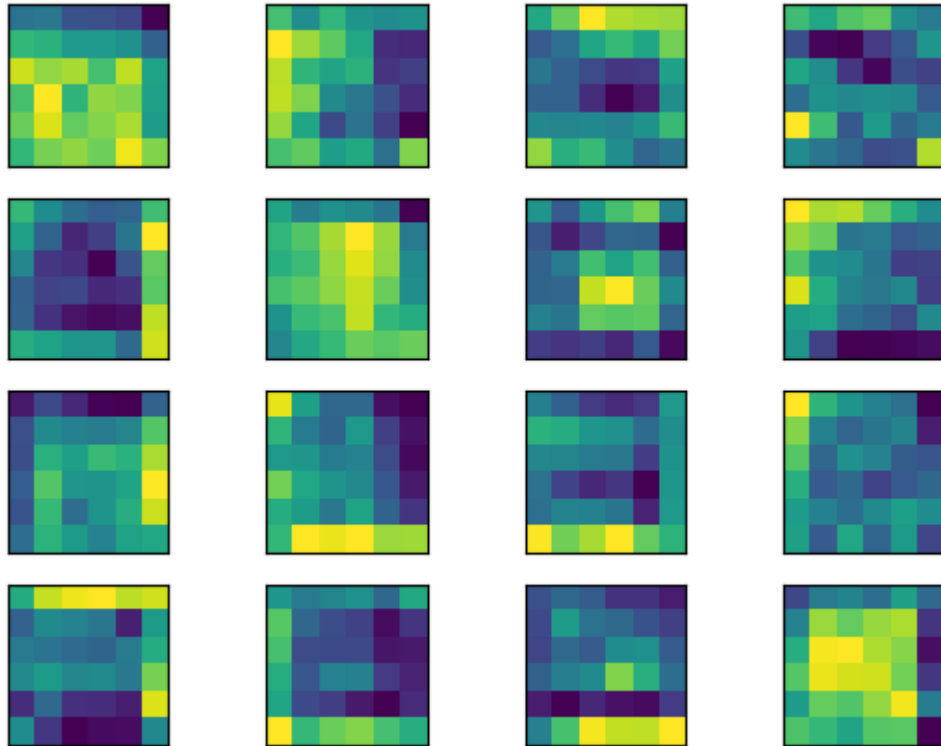
40번째 계층에 대한 특성 맵

```
result = LayerActivations(model.features, 40)
model(img)
activations = result.features
```

특성 맵 확인

```
fig, axes = plt.subplots(4, 4)
fig = plt.figure(figsize = (12, 8))
fig.subplots_adjust(left = 0, right = 1, bottom = 0, top = 1, hspace = 0.05, wspace = 0.05)

for row in range(4):
    for column in range(4):
        axis = axes[row][column]
        axis.get_xaxis().set_ticks([])
        axis.get_yaxis().set_ticks([])
        axis.imshow(activations[0][row * 10 + column])
plt.show()
```

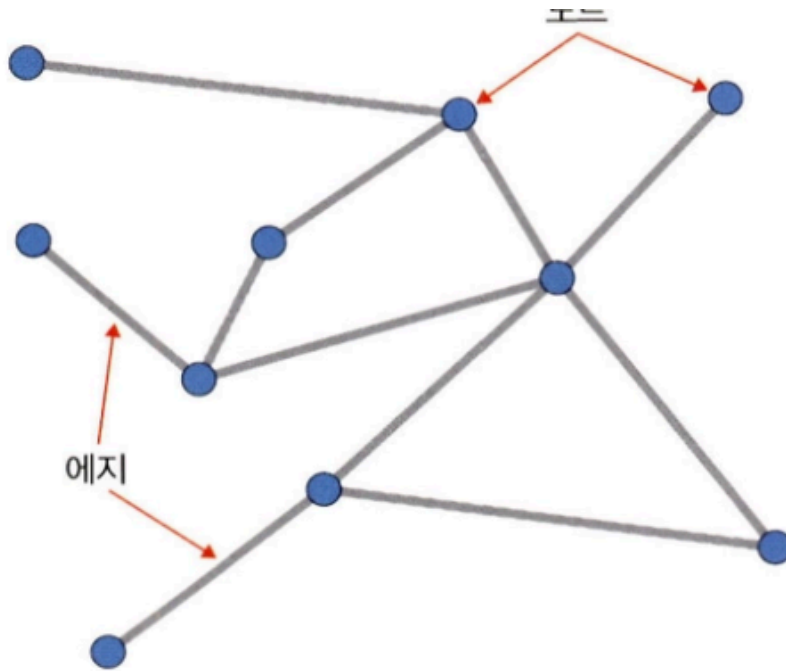
- 이제 원래 입력 이미지에 대한 형태는 전혀 찾아볼 수 없음
- 즉, 출력층에 가까울수록 원래 형태는 찾아볼 수 없고, 이미지 특징들만 전달됨

5.5 그래프 합성곱 네트워크 (GCN)

- GCN(graph convolutional graph)은 그래프 데이터를 위한 신경망

5.5.1 그래프란

- 방향성이 있거나 없는 엣지로 연결된 노드들의 집합

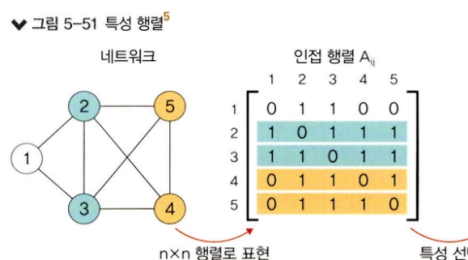


- 노드와 에지는 일반적으로 풀고자하는 문제에 대한 전문가 지식이나 직관 등으로 구성됨
- **노드**: 파란색 원 (원소들)
- **엣지**: 두 노드를 연결하는 선 (결합 방법 (single, double, triple, aromatic etc))

5.5.2 그래프 신경망 (GNN)

- 그래프 구조에서 사용하는 신경망으로, 그래프 데이터에 대한 표현은 두 단계로 이루어짐

1. 인접 행렬 (adjacent matrix)



- 네트워크의 노드 n 개를 $n \times n$ 행렬로 표현
- 이렇게 생성된 인접 행렬 내의 값 A_{ij} 는 i 와 j 의 관련성 여부를 만족하는 값으로 채움

- 컴퓨터가 이해하기 쉽게 그래프를 표현하는 과정

2. 특성 행렬(feature matrix)

▼ 그림 5-51 특성 행렬⁵

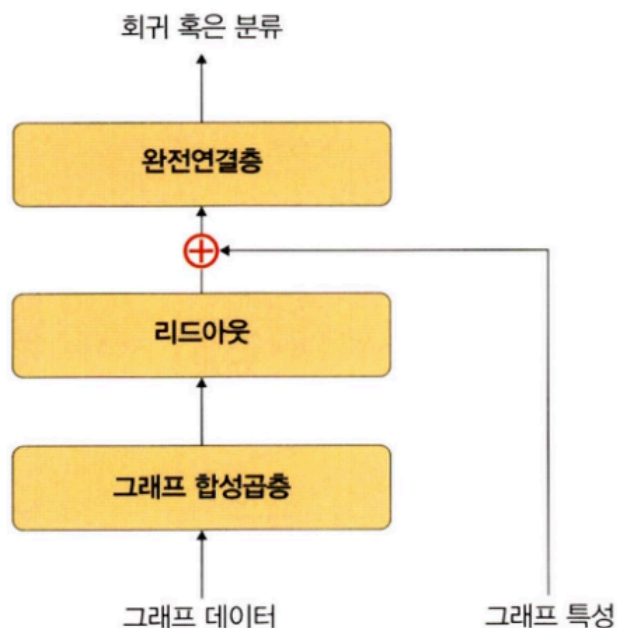


- 인접 행렬만으로는 특성을 파악하기 어렵기 때문에 단위 행렬을 적용함
- 각 입력 데이터에서 이용할 특성을 선택
- 특성 행렬에서 각 행은 선택된 특성에 대해 각 노드가 갖는 값을 의미함 (첫번째 행은 첫 번째 노드의 특성값)

5.5.3 그래프 합성곱 네트워크 (GCN)

- 이미지에 대한 합성곱을 그래프 데이터로 확장한 알고리즘

▼ 그림 5-52 그래프 합성곱 네트워크



- **리드아웃 (readout)**
 - 특성 행렬을 하나의 벡터로 변환하는 함수
 - 전체 노드의 특성 벡터에 대해 평균을 구하고 그래프 전체를 표현하는 하나의 벡터를 생성
- **그래프 합성곱층 (graph convolutional layer)**
 - GCN에서 가장 중요한 부분
 - 그래프 합성곱층을 이용한 그래프 형태의 데이터는 데이터로 변환되어 딥러닝 알고리즘을 적용할 수 있음 (기존 그래프 형태의 데이터로는 딥러닝 알고리즘 적용 불가)
- GCN은 SNS 관계 네트워크, 학술 연구에서 인용 네트워크, 3D mesh 같은 곳에서도 활용 가능